FoodShare – Technical Requirements Document (TRD)

Oct 22, 2026 · @momo

System overview

FoodShare is one Node.js service that exposes a REST API and a Socket.io gateway over a single MongoDB database; a web client uses both.

- **Web client** renders the nearby feed, role dashboards and the impact page, and subscribes to live events.
- **REST API (Express)** handles auth, listings, status transitions and analytics. Every write checks the caller's role.
- **Socket.io gateway** pushes new listings and status changes to connected clients. It reuses the same JWT as the REST API.
- **MongoDB** stores users and listings, with a 2dsphere index for radius queries and aggregation pipelines for analytics.

Tech stack

The stack is JavaScript end to end, so one team can own every layer.

Layer	Choice	Why
Runtime	Node.js 20 LTS	Single language, strong WebSocket support
API	Express 4	Simple, well-known, easy to test
Database	MongoDB with Mongoose 8	Native GeoJSON, `$near` and aggregation pipelines
Real-time	Socket.io 4	Rooms, reconnection and fallbacks built in
Auth	JWT (jsonwebtoken) and bcryptjs	Stateless sessions shared by REST and sockets

Layer	Choice	Why
Web client	React with Vite	Component model fits role dashboards
Maps	Leaflet with OpenStreetMap tiles	No API key or per-view cost
Charts	Chart.js or Recharts	Impact dashboard charts
Tests	Jest and Supertest	API and transition tests
Hosting	Docker on Render or Railway, MongoDB Atlas	Low-ops pilot deployment

API design

All endpoints live under `/api` and, except register and login, need an `Authorization: Bearer <token>` header. A wrong role returns 403; a transition from the wrong state returns 409.

Method and path	Role	Purpose
POST /auth/register	Public	Create account with role donor, volunteer or receiver
POST /auth/login	Public	Return JWT and user
GET /auth/me	Any	Current user
POST /listings	Donor	Create listing with lat, lng, quantity, expiry
GET /listings/nearby	Any	Available listings within `radiusKm` (default 5) of `lat`, `lng`
GET /listings/mine	Any	Caller's own listings by role
GET /listings/pickups	Volunteer	Claimed listings waiting for pickup, nearest first
POST /listings/:id/claim	Receiver	Available to Claimed
POST /listings/:id/pickup	Volunteer	Claimed to In Transit

Method and path	Role	Purpose
POST /listings/:id/deliver	Volunteer (assigned)	In Transit to Delivered
POST /listings/:id/cancel	Donor (owner)	Available to Cancelled
GET /analytics/impact	Any	Meals saved, CO2 avoided, breakdowns; `scope=me` for personal view

Errors use `{ "error": "message" }` with 400 validation, 401 unauthenticated, 403 wrong role, 409 state conflict.

Real-time and geospatial design

Geospatial. Each listing stores a GeoJSON Point as `[longitude, latitude]` with a 2dsphere index. The nearby feed uses `$near` with `$maxDistance` in metres (radius in km times 1000), which also sorts by distance.

Real-time. Clients connect to Socket.io with the JWT in the handshake; invalid tokens are rejected. Every client joins the `listings` room, and can join `listing:<id>` to follow one item.

Event	Sent to	Payload
`listing:new`	`listings` room	Full listing with donor info
`listing:update`	`listings` room	Full listing after claim, pickup, delivery or cancel
`listing:status`	`listing:<id>` room	id, status, updatedAt

Race conditions. Two receivers may claim the same listing at once. Each transition is one atomic `findOneAndUpdate` filtered on the expected current status, so only one request succeeds and the other gets 409.

Client behaviour. The REST response is the source of truth; socket events update the screen. On reconnect the client refetches the nearby feed to catch missed events.

Security, performance and observability

- **Auth:** bcrypt-hashed passwords, 7-day JWT, role claim re-checked against the database on each request.
- **Input safety:** validate every body; whitelist fields on create; never accept `role`, `status` or `claimedBy` from the client on listings.
- **Abuse control:** rate limit auth and write endpoints; add a CAPTCHA on registration if spam appears.
- **Transport:** HTTPS only; CORS restricted to the web client's origin in production.
- **Secrets:** `JWT_SECRET` and `MONGO_URI` come from environment variables, never from the repository.
- **Performance:** 2dsphere index plus an index on `status`; cap nearby results at 100; paginate list endpoints.
- **Scalability:** the Node service is stateless except sockets; add the Socket.io Redis adapter before running more than one instance.
- **Observability:** structured request logs, error tracking, and a health endpoint for uptime checks.

Deployment and testing

Environments. Local (Docker Compose with MongoDB), staging and production, each with its own database and secrets.

Configuration. `PORT`, `MONGO_URI`, `JWT_SECRET`, `CO2_PER_KG` and `KG_PER_MEAL` are read from the environment.

Testing

- Unit tests for the transition rules and the impact calculation.
- API tests with Supertest against an in-memory MongoDB for each endpoint and role.
- A concurrency test: two simultaneous claims on one listing must produce one 200 and one 409.
- A geo test: listings at 4 km and 6 km, query radius 5 km, only the first appears.
- Socket test: a status change reaches a second connected client.

Release. CI runs tests on each push; merges to main deploy to staging, and a manual approval promotes to production.